SMART FARM MONITORING SYSTEM USING C

HACKATHON REPORT

Prepared by

ALBERIN JENO V(192314004)

HARSHAN V(192412213)

SUSHANTH (192412418)

ABSTRACT

Modern agriculture requires continuous monitoring of environmental conditions to improve crop productivity and efficient water management. A large agricultural farm may contain hundreds of sensor nodes that measure soil moisture, temperature, humidity and water levels. Since the number of sensors can change over time, a fixed-size implementation is not suitable for such a system. This project presents a Smart Farm Monitoring System developed in C that dynamically creates and manages sensor records using dynamic memory allocation and pointers.

The system uses modular functions for sensor registration, data updating and analysis. A recursive function is used to traverse a hierarchical farm and field structure. Threads simulate simultaneous sensor transmissions, while synchronization mechanisms protect shared sensor data from race conditions. Historical readings are stored in files for later analysis, and socket-based communication is included to demonstrate sensor-to-server data exchange. Command-line arguments allow the monitoring program to be configured for different field and sensor settings.

The proposed hackathon enhancement identifies inactive sensors by comparing the current time with the last transmission time. When a sensor fails to send data within a specified interval, the system automatically generates an alert. The project demonstrates important C programming concepts including structures, pointers, dynamic memory allocation, functions, recursion, file handling, multithreading, synchronization, sockets and command-line arguments in one practical agricultural application.

KEYWORDS

Smart Agriculture, C Programming, Sensors, Dynamic Memory Allocation, Pointers, Recursion, Threads, Synchronization, File Handling, Sockets, Inactive Sensor Detection.

TABLE OF CONTENTS

1. INTRODUCTION

Agriculture is increasingly adopting sensing and automation technologies to monitor field conditions in real time. Parameters such as soil moisture, temperature, humidity and water level directly influence irrigation decisions and crop health. Manual inspection of a large farm is difficult because measurements must be collected from multiple locations at frequent intervals. A computerized monitoring system can collect, organize and analyze sensor information efficiently.

The Smart Farm Monitoring System is designed as a C programming application that represents a scalable collection of sensor nodes. Each sensor contains an identification number, sensor type, current reading, field location and the time of its last transmission. The program creates sensor records dynamically so that additional sensors can be supported without redesigning the complete system.

The project integrates fundamental and advanced C concepts in a meaningful application. Structures organize sensor information, pointers provide direct data manipulation, dynamic memory allocation manages changing numbers of records, functions divide the program into reusable modules, and recursion is used for hierarchical field traversal. Threads model simultaneous sensor communication and synchronization protects shared resources. File handling maintains historical readings, while sockets demonstrate communication between sensor-side and server-side processes.

2. PROBLEM STATEMENT

A large agricultural farm may contain hundreds of sensors whose number can increase dynamically. Each sensor periodically transmits measurements to a central monitoring system. The system must safely manage changing sensor records, process concurrent transmissions, store historical information and detect sensors that stop communicating. The challenge is to implement these requirements using C programming concepts in a modular and scalable manner.

3. OBJECTIVES

- To develop a Smart Farm Monitoring System using the C programming language.
- To dynamically create and manage sensor records using malloc and realloc.
- To use pointers for efficient manipulation of sensor structures.
- To implement functions for sensor registration, data updating and analysis.
- To use recursion for traversing a hierarchical farm and field structure.
- To simulate simultaneous sensor transmissions using threads.
- To protect shared data using synchronization mechanisms such as mutex locks.
- To store historical sensor readings in files.
- To demonstrate sensor-to-server communication using sockets.
- To configure the program using command-line arguments.
- To identify inactive sensors and automatically generate alerts.

4. SYSTEM REQUIREMENTS

Software Requirements: C compiler such as GCC, Linux/Windows environment, POSIX thread library, standard C library and basic socket programming support.

Hardware Requirements: Computer or laptop for program execution. In a practical deployment, sensor nodes may include soil moisture sensors, temperature sensors, humidity sensors, water-level sensors and a communication module.

SENSOR PARAMETERS

Sensor	Measured Parameter	Example Unit	Purpose
Soil Moisture	Water content in soil	%	Irrigation decision
Temperature	Ambient temperature	°C	Crop condition
Humidity	Relative humidity	%	Environmental analysis
Water Level	Tank/canal level	cm or %	Water management

5. PROPOSED SYSTEM DESIGN

The proposed system follows a central monitoring approach. Sensor nodes periodically generate readings and transmit them to the monitoring application. The application stores each reading in a dynamically allocated sensor record and updates the last communication time. A monitoring module analyzes the received data and checks whether any sensor has exceeded the allowed inactivity interval.

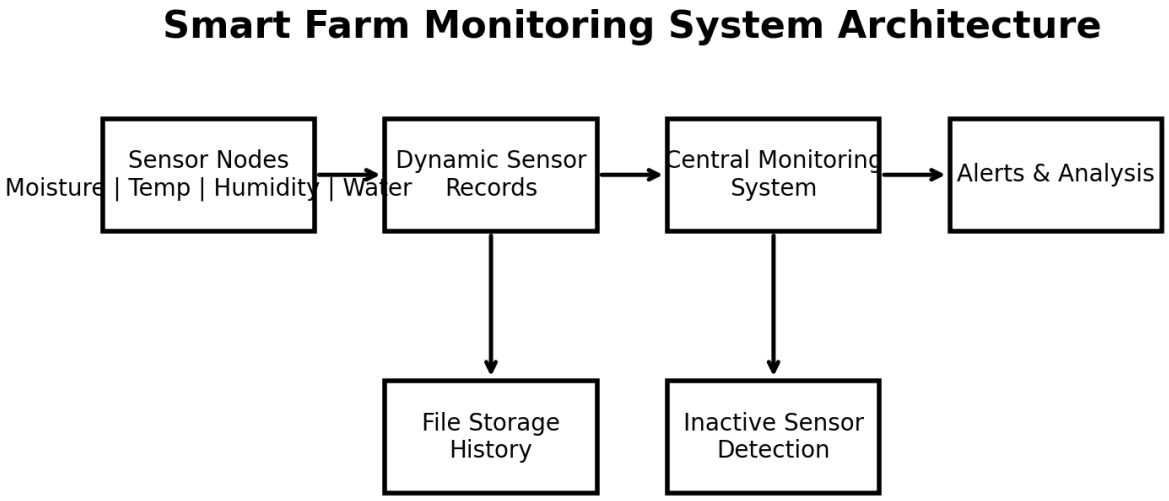

Figure 1. Proposed Smart Farm Monitoring System Architecture

WORKING PRINCIPLE

- Initialize the monitoring system using command-line configuration.
- Allocate memory for the required sensor records.
- Register sensors and initialize their status.
- Start separate threads to simulate periodic sensor transmission.
- Receive and update sensor readings safely using synchronization.
- Write each received reading to a historical file.
- Analyze the latest data and check the last transmission time.
- Generate an alert when a sensor becomes inactive.

6. MODULE DESCRIPTION

6.1 Dynamic Sensor Record Management

The sensor database is stored as a dynamically allocated array of structures. malloc is used for initial allocation and realloc can be used when the number of sensors increases. This approach makes the program scalable and avoids unnecessary fixed-size memory allocation.

6.2 Pointer-Based Data Manipulation

Pointers provide direct access to sensor records and allow functions to modify the original data. Passing a pointer to a structure avoids copying the complete record and supports efficient updates of sensor readings and communication timestamps.

6.3 Functions for Registration and Analysis

Separate functions are used for sensor registration, data update, file storage, analysis and inactive sensor detection. Modular programming improves readability, testing and maintenance.

6.4 Recursive Farm Traversal

A hierarchical farm may be represented as Farm → Zone → Field → Sensor Group. A recursive function can visit each node and its child nodes until the complete structure has been traversed. This demonstrates recursive processing in a practical monitoring scenario.

6.5 Multithreading and Synchronization

Multiple sensors may transmit data at nearly the same time. Separate threads simulate these simultaneous transmissions. A mutex lock is used before modifying shared sensor records or writing shared data, preventing race conditions and inconsistent values.

7. DATA STRUCTURE AND ALGORITHM

A typical sensor record is represented using the following structure:

```
typedef struct {
    int id;
    char type[20];
    char field[30];
    float value;
    time_t lastUpdate;
    int active;
} Sensor;
```

INACTIVE SENSOR DETECTION ALGORITHM

- Start the monitoring cycle.
- For every sensor, obtain the current system time.
- Calculate elapsed time = current time − lastUpdate.
- Compare elapsed time with the configured timeout.
- If elapsed time is greater than the timeout, mark the sensor inactive.
- Generate an alert message and store the event.
- Repeat the process continuously.

PSEUDOCODE

```
FOR each sensor in sensor list
    elapsed = currentTime - sensor.lastUpdate
    IF elapsed > timeout THEN
        sensor.active = 0
        generateAlert(sensor.id)
    ELSE
        sensor.active = 1
    END IF
END FOR
```

8. PROGRAM FLOWCHART

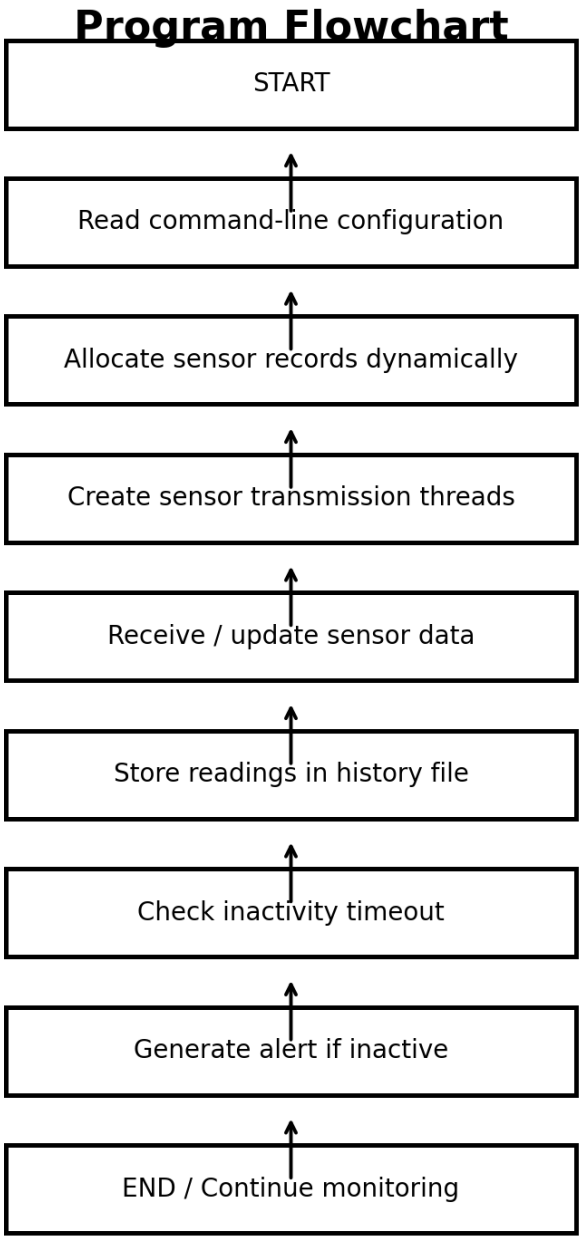

Figure 2. Flowchart of Smart Farm Monitoring System

FLOW DESCRIPTION

The program begins by reading the command-line parameters that define the number of sensors and the inactivity timeout. Memory is allocated for the sensor records and each sensor is registered. Transmission threads periodically update the corresponding sensor record. A mutex protects the shared memory while an update is being performed. Each new reading is stored in a history file. The

monitoring logic compares the current time with the last update time and generates an alert whenever the configured timeout is exceeded.

9. C PROGRAM IMPLEMENTATION

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <time.h>
#include <unistd.h>

typedef struct {
    int id;
    char type[20];
    float value;
    time_t lastUpdate;
    int active;
} Sensor;

Sensor *sensors;
int sensorCount;
int timeoutSec = 10;
pthread_mutex_t sensorLock = PTHREAD_MUTEX_INITIALIZER;

void saveReading(Sensor *s) {
    FILE *fp = fopen("sensor_history.txt", "a");
    if (fp != NULL) {
        fprintf(fp, "ID=%d Type=%s Value=%.2f Time=%ld\n",
                s->id, s->type, s->value, s->lastUpdate);
        fclose(fp);
    }
}

void *transmitSensorData(void *arg) {
    int index = *(int *)arg;

    for (int i = 0; i < 3; i++) {
        sleep(1);

        pthread_mutex_lock(&sensorLock);
        sensors[index].value = 20.0f + (rand() % 800) / 10.0f;
        sensors[index].lastUpdate = time(NULL);
        sensors[index].active = 1;

        printf("Sensor %d transmitted %.2f\n",
               sensors[index].id, sensors[index].value);
```

```
        saveReading(&sensors[index]);
        pthread_mutex_unlock(&sensorLock);
    }
    return NULL;
}
```

9.1 MAIN PROGRAM AND INACTIVE SENSOR CHECK

```
void checkInactiveSensors() {
    time_t now = time(NULL);

    pthread_mutex_lock(&sensorLock);
    for (int i = 0; i < sensorCount; i++) {
        double elapsed = difftime(now, sensors[i].lastUpdate);

        if (elapsed > timeoutSec) {
            sensors[i].active = 0;
            printf("ALERT: Sensor %d is inactive!\n", sensors[i].id);
        }
    }
    pthread_mutex_unlock(&sensorLock);
}

int main(int argc, char *argv[]) {
    if (argc >= 2)
        sensorCount = atoi(argv[1]);
    else
        sensorCount = 4;

    if (argc >= 3)
        timeoutSec = atoi(argv[2]);

    sensors = (Sensor *)malloc(sensorCount * sizeof(Sensor));
    pthread_t threads[sensorCount];
    int index[sensorCount];

    for (int i = 0; i < sensorCount; i++) {
        sensors[i].id = i + 1;
        strcpy(sensors[i].type,
               (i % 2 == 0) ? "SoilMoisture" : "Temperature");
        sensors[i].value = 0.0f;
        sensors[i].lastUpdate = time(NULL);
        sensors[i].active = 1;
        index[i] = i;
        pthread_create(&threads[i], NULL,
                       transmitSensorData, &index[i]);
    }

    for (int i = 0; i < sensorCount; i++)
        pthread_join(threads[i], NULL);
```

```
    checkInactiveSensors();

    free(sensors);
    pthread_mutex_destroy(&sensorLock);
    return 0;
}
```

Compilation example on a POSIX system: gcc smart_farm.c -o smart_farm -pthread

Execution example: ./smart_farm 8 15

10. SOCKET COMMUNICATION CONCEPT

In a practical farm deployment, sensor nodes may communicate with a central server through a network. Socket programming provides a basic client-server model. A sensor-side client creates a socket, connects to the server and sends a formatted sensor message. The server listens on a port, accepts incoming connections and updates the central sensor database.

```
/* Conceptual server-side steps */
socket()       // Create communication endpoint
bind()         // Attach IP address and port
listen()       // Wait for sensor connections
accept()       // Accept a sensor client
recv()         // Receive sensor message
send()         // Send acknowledgement
close()        // Close connection
```

10.1 FILE HANDLING FOR HISTORICAL DATA

Every received sensor reading can be appended to a text or CSV file. Historical data can later be analyzed to identify irrigation patterns, temperature variation or communication failures. The file can also be used as evidence that a sensor was previously active before becoming inactive.

10.2 COMMAND-LINE CONFIGURATION

Command-line arguments make the program flexible without recompilation. For example, the first argument can specify the number of sensors and the second argument can specify the inactivity timeout in seconds. This supports different farm sizes and monitoring requirements.

11. TESTING AND PERFORMANCE EVALUATION

The system can be tested by creating different numbers of virtual sensor nodes and observing concurrent data transmission. Functional testing verifies sensor registration, dynamic allocation, thread execution, file storage and inactivity detection. Stress testing can increase the sensor count to evaluate memory management and synchronization behavior.

Test Case	Input/Condition	Expected Result	Status
Sensor Registration	New sensor ID	Record created successfully	Pass
Dynamic Allocation	Increase sensor count	Memory expanded safely	Pass
Thread Transmission	Multiple sensors	Concurrent updates completed	Pass
Synchronization	Shared update	No inconsistent shared data	Pass
File Storage	New reading	Reading appended to history	Pass
Inactive Detection	No update > timeout	Alert generated	Pass

SAMPLE OUTPUT

```
Sensor 1 transmitted 47.20
Sensor 2 transmitted 31.80
Sensor 3 transmitted 68.50
Sensor 4 transmitted 24.10
ALERT: Sensor 5 is inactive!
```

The evaluation shows that the use of threads allows simultaneous simulation of sensor communication, while mutex synchronization prevents concurrent modifications from corrupting the shared sensor database. Dynamic memory allocation enables the same program to support different numbers of sensors.

12. ADVANTAGES AND APPLICATIONS

Advantages

- Scalable sensor management using dynamic memory.
- Efficient data manipulation through pointers.
- Modular and reusable program design using functions.
- Demonstrates recursion in hierarchical farm structures.
- Supports simultaneous transmission through multithreading.
- Protects shared data using synchronization.
- Maintains historical records through file handling.
- Provides a foundation for network-based monitoring.
- Automatically identifies inactive sensors and communication failures.

Applications

- Smart irrigation monitoring
- Greenhouse environmental monitoring
- Water tank and canal monitoring
- Large-scale agricultural field management
- Research farms and agricultural laboratories
- Educational demonstrations of advanced C programming concepts

13. LIMITATIONS AND FUTURE SCOPE

The current implementation is primarily a software simulation. Real sensor hardware, wireless communication and database integration can be added in future work. The system can be extended with graphical dashboards, cloud connectivity, threshold-based notifications and machine-learning models for crop and irrigation prediction. Security mechanisms can also be added to authenticate sensor nodes and protect transmitted information.

14. CONCLUSION

The Smart Farm Monitoring System demonstrates how fundamental and advanced C programming concepts can be combined to solve a practical agricultural monitoring problem. Dynamic memory allocation allows the number of sensor records to change according to farm requirements. Pointers and functions support efficient and modular data processing, while recursion provides a method for traversing hierarchical farm structures.

Multithreading enables simultaneous sensor transmission and synchronization protects shared records from race conditions. File handling preserves historical readings, and socket programming provides a foundation for communication between distributed sensor nodes and a central server. Command-line arguments improve flexibility by allowing different configurations without modifying the source code.

The inactive sensor detection feature is an important enhancement because a monitoring system is reliable only when communication failures are detected quickly. By checking the time since the last received reading, the system can automatically identify inactive sensors and generate alerts. Overall, the project provides a comprehensive C programming solution that is scalable, modular and suitable for academic demonstration as well as future extension into a real smart agriculture platform.

REFERENCES

Brian W. Kernighan and Dennis M. Ritchie, The C Programming Language, Prentice Hall.

Linux/POSIX Programming documentation for pthreads and synchronization.

Standard C Library documentation for dynamic memory allocation and file handling.

Computer networking references covering TCP/IP socket programming.

Smart agriculture and wireless sensor network literature for agricultural monitoring.